RV UNIVERSITY

Go, change the world

an initiative of RV EDUCATIONAL INSTITUTIONS

New-Age Global
University for
Liberal Education

School of Computer Science and Engineering

Where Ideas Ignite Minds

Laboratory Record Book

Programming in C

CS1003

BTech Computer Science (Hons.)

Academic Year 2025-26

Rashtreeya Sikshana Samithi Trust

RV UNIVERSITY

School of Computer Science and Engineering

Bengaluru – 560059

PROGRAMMING IN C

COURSE CODE: CS1003

I SEMESTER B.Tech (Hons.)

LABORATORY RECORD 2025-2026

RV UNIVERSITY

School of Computer Science and Engineering

Bengaluru – 560 059

LABORATORY CERTIFICATE

This is to certify that Mr./Ms. _____ has satisfactorily completed the course of experiments in Practical ***Programming in C*** **(CS1003)** prescribed by the **School of Computer Science and Engineering** during the year **2025-26.**

USN: _____ Semester: _____

Marks	
Maximum	**Obtained**
25	

Signature of Faculty In-charge **Program Director**

Date:

Vision and Mission of the School of Computer Science and Engineering

Vision

To be a pioneering school of Computer Science and Engineering committed to fostering liberal education and empowering the next generation of technologists to make a positive global socio-economic impact.

Mission

- To be a pioneer in computer science education, fostering multidisciplinary research, innovation and entrepreneurship.
- To provide state-of-the-art facilities and dynamic curriculum that enables exemplary pedagogy, integrating advanced technology, theoretical foundations, and hands-on applications to address real-world challenges.
- To promote diverse communities of faculty and students through national and international academic, industry collaborations contributing to institution-building.
- To cultivate a generation of ethical, self-motivated, and empathetic problem solvers dedicated to achieving sustainable development goals.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Graduates will be able to demonstrate excellence in the field of Computer Science and Engineering through advanced research and entrepreneurship.

PEO2: Graduates will be able to achieve excellence in innovation through national and international academic and industry collaborations.

PEO3: Graduates will be able to actively engage in cutting-edge interdisciplinary and multidisciplinary research.

PEO4: Graduates will be able to function effectively as talented pool of ethical, sustainable, self-driven and life-long empathetic problem solvers.

PROGRAM OUTCOMES (POs)

Engineering Graduates will be able to:

PO1: **Application of Knowledge.** Apply knowledge from the core/interdisciplinary areas and develop skills to solve complex real-world problems.

PO2: **Critical Thinking & Creativity.** Analyse and evaluate the existing situations, practices and evidence to develop innovative multi-perspective solutions.

PO3: **Research & Innovation.** Cultivate a keen sense of observation and enquiry using appropriate methodology to develop new ideas, processes & solutions in response to contemporary challenges.

PO4: **Digital Transformation.** Instill an ability for confident, critical and responsible use of digital technologies for learning, work and participation in society.

PO5: **Communication & Interpersonal Skills.** Develop skills to effectively express thoughts and ideas both orally and in writing to people from all sections of the society.

PO6: **Leadership & Collaborative Teamwork.** Exhibit skills to set goals and guide people to align effectively in building a team to achieve the goals.

PO7: **Project Management & Entrepreneurial Mindset**. Foster an enterprising spirit by working on diverse projects, demonstrating time management, resource allocation and risk assessment capabilities to create value.

PO8: **Lifelong Learning.** Engage in continuous self-directed learning to adapt to evolving technologies, societal changes and emerging global challenges.

PO9: **Global Perspective & Cultural awareness.** Acquire knowledge about the values and beliefs of diverse cultures, effectively engaging with empathy.

PO10: Ethical Reasoning & Social Responsibility. Demonstrate moral and value-based principles while being socially conscious and accountable.

PO11: Civic Engagement & Community Service. Participate actively in civic life exhibiting a commitment to community service, social justice and the common good.

PO12: Environmental Sustainability. Show responsibility to manage and work towards achieving the Sustainable Development Goals (SDGs)

Course Outcomes: After completing the course, the students will be able to:	
CO1	Understand and analyze the concepts of number system, memory, fundamental concepts in C programming language.
CO2	Demonstrate fundamental data types, operators and console I/O functions in C.
CO3	Implement programs using decision control statements, control flow statements, arrays, strings, functions, and pointers to solve mathematical problems.
CO4	Analyze and evaluate the programming concepts based on user defined data types and file handling in C language.
CO5	Design and implement appropriate concepts of C to solve real-world problems.

LIST OF PROGRAMS

1.	Write a program using function-like macros to add two numbers.
2.	Write a program to swap two numbers with and without using a third variable.
3.	Write a program to create a menu-driven basic arithmetic calculator.
4.	a) Write a program to print a Fibonacci series. b) Write a program to compute GCD of two numbers.
5.	a) Write a program to print nth term of a Fibonacci series using function. b) Write a program to compute GCD of two numbers using function.
6.	a) Write a program to print a Fibonacci series using recursion. b) Write a program to compute GCD of two numbers using recursion.
7.	Write a menu driven program to demonstrate the following array operations using functions: Insert, Delete, Update element at index i,
8.	Write a program to implement different string operations using functions.
9.	Write a program using structures to read, write and compute average-marks of n students. Take student name, USN, and marks in 5 subjects as input. Pass structure by reference to a function to update marks in one of the five subjects as given by user for a particular student. Finally display the new average for that student.
10.	Write a program to demonstrate different file operations in C.

Lab Assessment (10 Marks per program)				
Criteria	**Measuring Methods**	**Excellent**	**Good**	**Poor**
Understanding of problem statement (2 Marks)	Observations	The student exhibits a thorough understanding of the requirements for the problem. **(2 M)**	The student has a sufficient understanding of the requirements for the problem. **(1 M)**	The student does not have a clear understanding of the requirements for the problem. **(0 M)**
Execution (2 Marks)	Observations	The Student demonstrates the execution of the program with optimized code and shows the output with appropriate validations and test cases handled. **(2 M)**	The Student demonstrates the execution of the program without optimization of the code and shows the output. **(1 M)**	The Student has not executed the program. **(0 M)**
Results and Documentation (2Marks)	Observations	Documentation with appropriate comments and output cases are noted in data sheets. **(2 M)**	Documentation with few comments and few output cases are noted in data sheets. **(1 M)**	Documentation with no comments and no output cases are noted in data sheets. **(0 M)**
Viva (4 Marks)				

INDEX

Sl. No	Program Name	Marks		Sign
		Record (6M)	Viva (4M)	
1.	Write a program using function-like macros to add two numbers.			
2.	Write a program to swap two numbers with and without using a third variable.			
3.	Write a program to create a menu-driven basic arithmetic calculator.			
4.	a) Write a program to print a Fibonacci series up to a given number of terms.			
	b) Write a program to compute GCD of two numbers.			
5.	a) Write a program to print nth term of a Fibonacci series using function.			
	b) Write a program to compute GCD of two numbers using function.			
6.	a) Write a program to print a Fibonacci series using recursion.			
	b) Write a program to compute GCD of two numbers using recursion.			
7.	Write a menu driven program to demonstrate the following array operations using functions: Insert, Delete, Update element at index i.			
8.	Write a program to implement different string operations using functions.			
9.	Write a program using structures to read, write and compute average-marks of n students. Take student name, USN, and marks in 5 subjects as input. Pass structure by reference to a function to update marks in one of the five subjects as given by user for a particular student. Finally display the new average for that student.			
10.	Write a program to demonstrate different file operations in C.			
	Total			
	Total (100 M)			
	Total (Scaled to 5 M)			

Record + Viva (5 Marks)	CIE3 Lab Internals (15 Marks)	CodeTantra Lesson Plan Completion (5 Marks)	Final Total (25 Marks)

Signature with date
(Faculty In-charge)

PROGRAM – 1

Pre-requisites:

● Data-types with their sizes, ranges, and format specifiers

Data Type	Size (bytes)	Range	Format Specifier
short int	2	-32,768 to 32,767	%hd
unsigned short int	2	0 to 65,535	%hu
unsigned int	4	0 to 4,294,967,295	%u
Int	4	-2,147,483,648 to 2,147,483,647	%d
long int	4	-2,147,483,648 to 2,147,483,647	%ld
unsigned long int	4	0 to 4,294,967,295	%lu
long long int	8	-(2^63) to (2^63)-1	%lld
unsigned long long int	8	0 to 18,446,744,073,709,551,615	%llu
signed char	1	-128 to 127	%c
unsigned char	1	0 to 255	%c
Float	4	1.2E-38 to 3.4E+38	%f
Double	8	1.7E-308 to 1.7E+308	%lf
long double	16	3.4E-4932 to 1.1E+4932	%Lf

● sizeof operator

Queries size of the object or type. Used when actual size of the object must be known.

Syntax:

1. sizeof(*type*) – Returns the size, in bytes, of the object representation of *type*.
2. sizeof(*type*) – Returns the size, in bytes, of the object representation of the type of *expression*. No implicit conversions are applied to *expression*.

Example: Sample output corresponds to a platform with 64-bit pointers and 32-bit int.

```
#include <stdio.h>
int main(void)
{   printf("sizeof(float)          = %zu\n", sizeof(float));
    printf("sizeof &main          = %zu\n", sizeof &main);
}
```

Possible output:

```
sizeof(float)       = 4
sizeof &main        = 8
```

Escape sequences

Escape sequence	Character represented
\a	Alert (Beep, Bell)
\b	Backspace
\f	Formfeed Page Break
\n	Newline (Line Feed)
\r	Carriage Return
\t	Horizontal Tab
\v	Vertical Tab
\\	Backslash
\'	Apostrophe or single quotation mark
\"	Double quotation mark
\?	Question mark (used to avoid trigraphs)

Macros

A *macro* is a fragment of code which has been given a name. Whenever the name is used, it is replaced by the contents of the macro. There are two kinds of macros. They differ mostly in what they look like when they are used. *Object-like* macros resemble data objects when used, *function-like* macros resemble function calls.

By convention, macro names are written in upper case. The macro's body ends at the end of the `#define' line. You may continue the definition onto multiple lines, if necessary, using backslash-newline. There is no restriction on what can go in a macro body provided it decomposes into valid preprocessing tokens. The C preprocessor scans your program sequentially. Macro definitions take effect at the place you write them. Therefore, the following input to the C preprocessor

```
foo = X;
#define X 4
bar = X;
```

produces

```
foo = X;
bar = 4;
```

Below is a macro that computes the minimum of two numeric values

```
#define min(X, Y)  ((X) < (Y) ? (X) : (Y))
  x = min(a, b);          ==>  x = ((a) < (b) ? (a) : (b));
  y = min(1, 2);          ==>  y = ((1) < (2) ? (1) : (2));
```

Program for Lab 1

Write a program using function-like macros to add two numbers.

Syntax:

#define MACRO_NAME(params) MACRO_BODY

where **MACRO_NAME** is name of the macro. **params** is parameters passed to macro. **MACRO_BODY** is the body which will have actual logic of macro.

Pseudocode:

User input: $num1, num2$

Output: sum

1. Define a macro named **ADD(x, y)** that computes the sum of x and y.
2. Set $sum = 0$.
3. Call the **ADD** macro with $num1$ and $num2$ as arguments and store the result in sum.
4. Display the result: "The sum of" $num1$ " and " $num2$ " is " sum.

Stick Data sheets of Program-1 here:

OUTPUT:

PROGRAM – 2

Pre-requisites:

● Arithmetic Operators.

Operator	Operator name	Example	Result
+	addition	a + b	the addition of a and b
-	subtraction	a – b	the subtraction of b from a
*	product	a * b	the product of a and b
/	division	a / b	the division of a by b
%	remainder	a % b	the remainder of a divided by b

● Relational or Comparison Operators.

Operator name	Syntax	Result
Equal to	a == b	Returns true if a is equal to b, false otherwise.
Not equal to	a != b	Returns true if a is not equal to b, false otherwise.
Less than	a < b	Returns true if a is less than b, false otherwise.
Greater than	a > b	Returns true if a is greater than b, false otherwise.
Less than or equal to	a <= b	Returns true if a is less than or equal to b, false otherwise.
Greater than or equal to	a >= b	Returns true if a is greater than or equal to b, false otherwise.

● Logical Operators.

Operator	Operator name	Example	Result
!	logical NOT	!a	the logical negation of a
&&	logical AND	a && b	the logical AND of a and b
||	logical OR	a || b	the logical OR of a and b

● Conditional/Ternary Operator.

Operator name	Syntax
conditional operator	a ? b : c

● Increment and Decrement Operators.

Operator name	Syntax
pre-increment	++a
pre-decrement	--a

Operator name	Syntax
post-increment	a++
post-decrement	a--

● Bit-wise Operators.

Operator	Operator name	Example	Result
~	bitwise NOT	~a	the bitwise NOT of a
&	bitwise AND	a & b	the bitwise AND of a and b
|	bitwise OR	a | b	the bitwise OR of a and b
^	bitwise XOR	a ^ b	the bitwise XOR of a and b
<<	bitwise left shift	a << b	a left shifted by b
>>	bitwise right shift	a >> b	a right shifted by b

● Compound Assignment Operators.

Operator name	Syntax
addition assignment	a += b
subtraction assignment	a -= b
multiplication assignment	a *= b
division assignment	a /= b
remainder assignment	a %= b
bitwise AND assignment	a &= b
bitwise OR assignment	a |= b
bitwise XOR assignment	a ^= b
bitwise left shift assignment	a <<= b
bitwise right shift assignment	a >>= b

● Type Conversion and Type Casting

Type conversion is the process of converting one data type to another only if the conversion is possible. There are two types of conversion:

- **Implicit/Automatic Type Conversion** – It is automatically done by the compiler without any external trigger from the user. In the presence of more than one data type in an expression type conversion (type promotion) takes place to avoid any loss of data. Though loss of information is possible sometimes, signs can be lost (when signed is implicitly converted to unsigned), and overflow can occur (when long is implicitly converted to float). Data type of the variables gets

upgraded to the data type of the variable with the largest data type. Here is the sequence from smallest to the largest data type:

bool -> char -> short int -> int -> unsigned int -> long ->

unsigned -> long long -> float -> double -> long double

Example:

```
#include <stdio.h>
int main()
{       int x = 10;
        char y = 'a';
        x = x + y;
        float z = x + 1.0;
        printf("x = %d, z = %f", x, z);
}
```

Output:

x = 107, z = 108.000000

- **Explicit Type Conversion/Type casting** – It is user-defined, i.e. the user can typecast the result to make it of a particular data type. Syntax: (type) expression, where type indicates the data type to which the final result is converted.

  Example:

```
#include <stdio.h>
int main()
{
        float a = 1.5;
        int b = (int)a;
        printf("a = %f\n", a);
        printf("b = %d\n", b);
}
```

Output:

a = 1.500000
b = 1

Program for Lab 2

Write a program to swap two numbers with and without using a third variable.

Pseudocode (with a third variable):

User input: $num1, num2$

Output: sum

1. Declare a variable, say $temp$.
2. Assign $num1$ to $temp$.
3. Assign $num2$ to $num1$.
4. Assign $temp$ to $num2$.
5. Display the result:

   "Value of num1 = " $num1$

   "Value of num2 = " $num2$

Pseudocode (without a third variable):

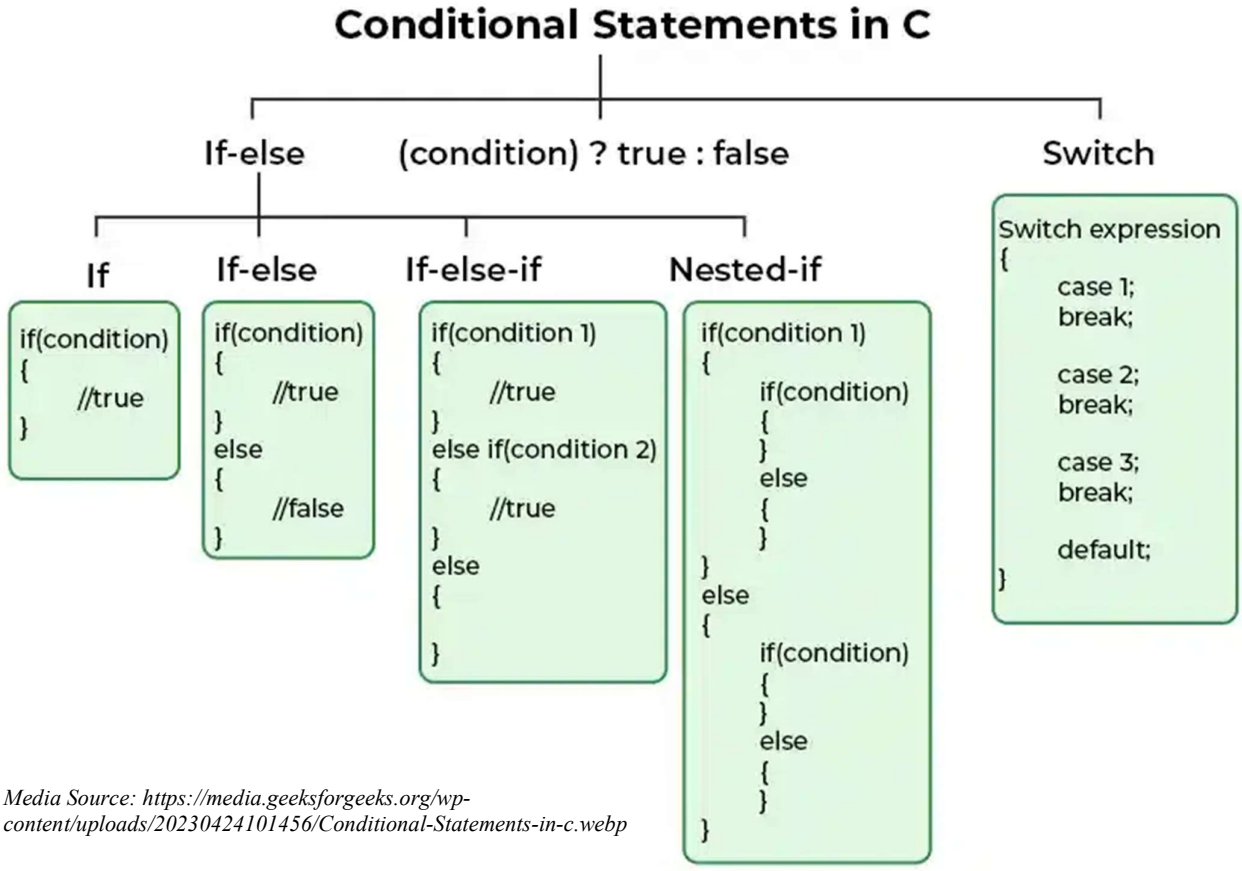
https://www.geeksforgeeks.org/swap-two-numbers-without-using-temporary-variable/

Stick Data sheets of Program-2 here:

OUTPUT:

PROGRAM – 3

Pre-requisites:



Media Source: https://media.geeksforgeeks.org/wp-content/uploads/20230424101456/Conditional-Statements-in-c.webp

Program for Lab 3

Write a program to create a menu-driven basic arithmetic calculator.

Pseudocode:

User input: $num1, num2$ and choice of operation, $choice$.

1. Display the menu with choices for operations – addition, subtraction, multiplication and division.
2. Read the user's $choice$.
3. Execute the block of code corresponding to $choice$ and display the result.

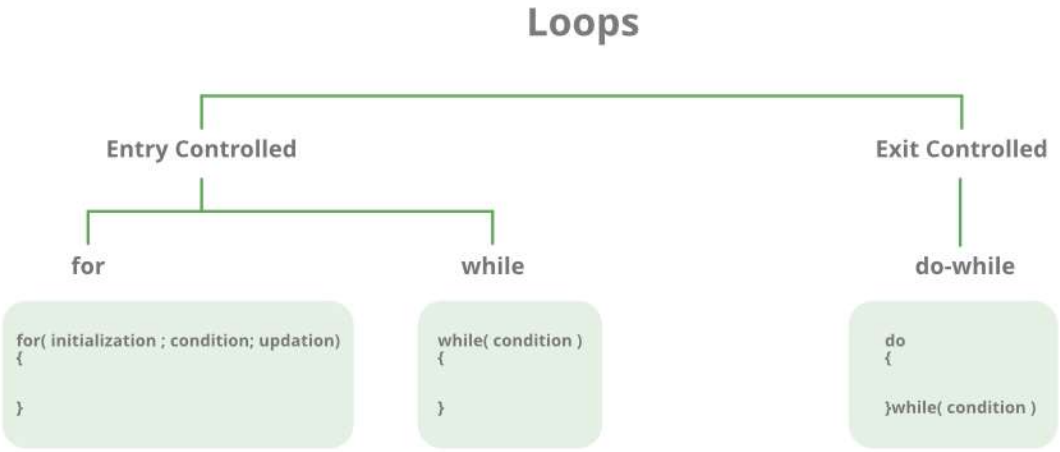
Note: The program should handle division by ZERO.

Stick Data sheets of Program-3 here:

OUTPUT

PROGRAM – 4

Pre-requisites:



Media Source: https://media.geeksforgeeks.org/wp-content/cdn-uploads/20191128194516/Cpp-loops.png

● Break and Continue

Break statement: This statement terminates the smallest enclosing loop. Syntax: break;

Continue statement: This statement skips the rest of the loop statement and starts the next iteration of the loop. Syntax: continue;

● Fibonacci Series/Sequence/Numbers

It is defined as the sequence of numbers in which each number in the sequence is equal to the sum of two numbers before it. The Fibonacci Sequence is given as: $0, 1, 1, 2, 3, 5, 8, 13, 21,$ …. The next number in sequence is obtained by adding the previous two terms, i.e., $13 + 21 = 34$.

● Greatest Common Divisor (GCD) / Highest Common Factor (HCF) of two Numbers

It is the greatest common factor that divides the two numbers perfectly.

Euclid's division algorithm:

It is stated only for positive integers. Let the positive integers be a and b, assuming $a > b$.

- Apply Euclid's division lemma to a and b and get two whole numbers q and r such that, $a = bq + r$; $0 < r < b$.
- If $r = 0$, then b is the HCF of a and b. If $r \neq 0$, then apply Euclid's division lemma to b and r.
- Continue the process until the remainder is zero.
- When the remainder is zero, the divisor at that stage is the GCD or HCF of the given numbers.

Programs for Lab 4

4a. Write a program to print a Fibonacci series up to a given number of terms.

Pseudocode:

User input: A positive integer, n.

- Declare and initialize two variables, $a = 0$ and $b = 1$.
- Declare a variable, c.
- Print the values of a and b.
- For each iteration from 3 to n.
  a. Compute the next Fibonacci number, i.e. $c = a + b$.
  b. Update a to the value of b.
  c. Update b to the value of c.
  d. Print the values of c.

4b. Write a program to compute GCD of two numbers.

Pseudocode:

User input: The two numbers, $num1$ and $num2$.

1. Declare and initialize a variable, $a = maximum\ of\ num1\ and\ num2$.
2. Declare and initialize a variable, $b = minimum\ of\ num1\ and\ num2$.
3. Declare and initialize a variable, $R = 0$.
4. Repeat (5) to (7) until R becomes $ZERO$.
5. Update R to the value of a % b.
6. Update a to the value of b.
7. Update b to the value of R.
8. Print the result:
   'GCD of ' $num1$ ' and ' $num2$ ' is ' a.

Stick Data sheets of Program-4 here:

OUTPUT:

PROGRAM – 5

Pre-requisites:

● Local and Global Variables

	Local	Global
Scope	Declared within a specific block of code, such as a function or loop. They are only accessible within the block in which they are declared.	Declared outside of any function or block of code, usually at the top of a program or in a separate file. They are accessible from any part of the program.
Lifetime	Exists only while the block is executing. Once the block of code in which they are declared exits, their memory is released, and they are no longer accessible.	Retain their value throughout the lifetime of the program unless explicitly modified or reset. They persist throughout the entire execution.
Name Conflicts	Can have the same name as variables in other blocks without conflict.	Should be used carefully to avoid unintended side effects, as they are accessible from anywhere.
Usage	Used for temporary storage or data relevant only within a specific context	Used for values that need to be accessed and modified by multiple parts of the program.

● Pointer

A derived data type that can store the address of other variables or a memory location. One can access and manipulate the data stored in that memory location using pointers. The size of pointers only depends on the system architecture and is independent of the type of data they are pointing to. Pointers should always be initialized to some value before their first usage. Otherwise, it may lead to number of errors.

- *Pointer Declaration*: Declare the pointer using dereference operator, i.e. *, before its name. Such an uninitialized pointer will point to some random memory address and is known as wild pointer.
- *Pointer Initialization*: Assign some initial value to the pointer variable, i.e. the memory address of a variable using the addressof (i.e., &) operator.

  Example:

```
int var = 10;
int * ptr;              // Pointer declaration
ptr = &var;             // Pointer initialization
// int *ptr = &var;     // Pointer definition: Declare and initialize the pointer in a single step.
```

- *Pointer Dereferencing*: Accessing the value stored in the memory address specified in the pointer using the dereferencing operator, i.e. *.

Following are the valid pointer arithmetic operations: Increment, Decrement, Addition or Subtraction of integer to a pointer, Subtraction or Comparison or Assignment of pointers of the same type.

Functions

The construct function associates the function body with the function name. C program begins execution from the main function, which either terminates, or invokes other, user-defined or library functions.

// Definition of a function with the name "sum" and with the body "{ return x + y; }"

```
int sum(int x, int y)
{
    return x + y;
}
```

A function declaration or a function definition introduces a function. Functions may accept zero or more *parameters*, which are initialized from the *arguments* during a function call, and may return a value to its caller by means of the return statement.

```
int n = sum(1, 2); // parameters x and y are initialized with the arguments 1 and 2
```

There are no nested functions. Each function definition must appear at file scope, and functions have no access to the local variables from the caller.

```
int sum(int, int); // function declaration
int main(void) // the main function definition
{   int x = 1;  // local variable in main
    sum(1, 2); // function call
}
int sum(int a, int b) // function definition
{
    //return x + a + b; //error: main's x is not available within sum
    return a + b;
}
```

The data passed when the function is being invoked is known as the Actual parameters. In the above program, 1 and 2 are known as actual parameters. Formal Parameters are the variable and the data type as mentioned in the function declaration. In the above program, a and b are known as formal parameters. One can pass arguments in two ways:

- *Pass by Value*: Values are copied from actual parameters into formal function parameters. As a result, any changes made inside the functions do not reflect in the caller's parameters.
- *Pass by Reference*: The caller's actual parameters and the function's formal parameters refer to the same memory locations, so any changes made inside the function are reflected in the caller's actual parameters.

Programs for Lab 5

5a. Write a program to find factorial of a number using function.

Pseudocode:

User input: num

1. Declare and initialize two variables, $fact = 1$ and $i = 1$.
2. Repeat until $i <= num$
   a. Update $fact = fact * i$
   b. Increment i by one.
3. Print the result:
   'Factorial of ' $num1$ ' is ' $fact$.

5b. Write a program to print nth term of a Fibonacci series using function.

Pseudocode:

User input: A positive integer, n.

1. Declare and initialize two variables, $a = 0$ and $b = 1$.
2. Declare a variable, c.
3. If $n = 1$, then print the value of a.
4. If $n = 2$, then print the value of b.
5. For each iteration from 3 to n.
   a. Compute the next Fibonacci number, i.e. $c = a + b$.
   b. Update a to the value of b.
   c. Update b to the value of c.
6. Print the result:
   n 'th Fibonacci term is ' c.

Stick Data sheets of Program-5 here:

OUTPUT:

PROGRAM – 6

Pre-requisites:

● Recursion

It is the process of a function calling itself repeatedly till the given condition is satisfied. A function that calls itself directly or indirectly is called a recursive function and such kind of function calls are called recursive calls. The basic syntax structure of the recursive functions is as follows:

```
function_return_type function_name (function_input_args) {
    // function statements
    // base condition
    // recursion case (recursive call)
}
```

Example:

```
int printSum(int n)
{   if(n == 0)
        return 0;
    else
        return n + printSum(n-1);
}
int main()
{   printf("%d\n",printSum(5));
}
```

Programs for Lab 6

6a. Write a program to print a Fibonacci series using recursion.

Base Case: $F_0 = 0$ and $F_1 = 0$

Recursive Formula: $F_n = F_{n-1} + F_{n-2}$

6b. Write a program to compute GCD of two numbers using recursion.

Pseudocode:

User input: $num1$ and $num2$

Function Name: $GCD(x, y)$

1. $Base\ Case: If\ y = 0\ then\ return\ x.$
2. **Recursive Formula:** Else call $GCD(y, x\ \%\ y)$

Stick Data sheets of Program-6 here:

OUTPUT:

PROGRAM – 7

Pre-requisites:

● Array Declaration

1D Array: `datatype arrayName [size];`

2D Array: `datatype arrayName [size1] [size2];`

3D Array: `datatype arrayName [size1] [size2] [size3];`

nD Array: `datatype arrayName [size1] [size2] ... [size`n`];`

● Array Initialization

- `datatype arrayName [size] = {value1, value2, ... value`N`};`
- `datatype arrayName [] = {value1, value2, ... value`N`};`
- Initialize array after declaration by individually assigning values to each element using loops.

● Array and Pointers

The array name is a constant pointer to the first element of the array and the array decomposes to the pointers when passed to the function. One can return an array from a function using a pointer to the first element of that array.

Program for Lab 7

Write a menu driven program to demonstrate the following array operations using functions: Insert, Delete, Update element at index i,

Let $A[MAX]$ be an array having N elements.

- **Pseudocode: INSERT**

  User input: A new value to be inserted, say $num1$ at index, say $index$.

  Function Name: $insert(A, N, MAX, num1, index)$

  1. $If\ N = MAX\ then\ return.$
  2. Else
     a. $N = N + 1$
     b. For all elements from $A[index]$ to $A[N]$ move to next adjacent location.
     c. $A[index] = num1$

- **Pseudocode: DELETE**

User input: An index, say $index$, whose value has to be deleted.

Function Name: $delete(A, N, index)$

1. $If\ N = 0\ then\ return.$
2. Else
   a. For all elements from $A[index + 1]$ to $A[N]$ move to previous adjacent location.
   b. $N = N - 1$

- **Pseudocode: UPDATE ELEMENT AT INDEX i**

User input: A new value to be updated, say $num1$ at index, say $index$.

Function Name: $update(A, N, num1, index)$

1. $If\ N = 0\ then\ return.$
2. Else $A[index] = num1$

Stick Data sheets of Program-7 here:

OUTPUT:

PROGRAM – 8

Strings in C

A String is a sequence of characters terminated with a null character '\0'. It is stored as an array of characters.

	Syntax
Declaration	`char str[size]; // size is some integer.`
Initialization	`char str[] = "ProgrammingInC";`
	`char str[20] = "ProgrammingInC";`
	`char str[12] = {'P','r','o','g','r','a','m','m','i','n','g','\0'};`
	`char str[] = {'P','r','o','g','r','a','m','m','i','n','g','\0'};`
Print on screen	`printf("%s",str);  OR  puts(str)`
Read from user	`scanf("%s",str); // till the whitespace is encountered.`
	`fgets(str,#numberOfCharacters,stdin) // to read a line of string.`
	`gets(str) // to read a line of characters from the standard input.`
	`scanf("%[^\n]s", str); // to read a line of string.`

string.h Header File

This header file defines several functions to manipulate C strings and arrays.

memcpy	Copy block of memory `void * memcpy (void * destination, const void * source, size_t num);`
memmove	Move block of memory `void * memmove (void * destination, const void * source, size_t num);`
Strcpy	Copy string

	`char * strcpy (char * destination, const char * source);`
strncpy	Copy characters from string `char * strncpy (char * destination, const char * source, size_t num);`
Strcat	Concatenate strings `char * strcat (char * destination, const char * source);`
Strncat	Append characters from string `char * strncat (char * destination, const char * source, size_t num);`
memcmp	Compare two blocks of memory `int memcmp (const void * ptr1, const void * ptr2, size_t num);`
strcmp	Compare two strings `int strcmp (const char * str1, const char * str2);`
strncmp	Compare characters of two strings `int strncmp (const char * str1, const char * str2, size_t num);`
memchr	Locate character in block of memory `const void * memchr (const void * ptr, int value, size_t num);`
strchr	Locate first occurrence of character in string `const char * strchr (const char * str, int character);`
strcspn	Get span until character in string `size_t strcspn (const char * str1, const char * str2);`
strpbrk	Locate characters in string `const char * strpbrk (const char * str1, const char * str2);`
strrchr	Locate last occurrence of character in string `const char * strrchr (const char * str, int character);`
strspn	Get span of character set in string `size_t strspn (const char * str1, const char * str2);`
strstr	Locate substring `const char * strstr (const char * str1, const char * str2);`
strtok	Split string into tokens `char * strtok (char * str, const char * delimiters);`
memset	Fill block of memory `void * memset (void * ptr, int value, size_t num);`

strerror	Get pointer to error message string `char * strerror (int errnum);`
strlen	Get string length `size_t strlen (const char * str);`

Program for Lab 8

Write a program to implement different string operations using functions.

Pseudocode:

Function Name: $stringLength(char\ str[n])$

1. Declare and initialize a variable, $length = 0$
2. While $str[length]$ is not $'\backslash 0'$
3. $length++$
4. Return $length$

Pseudocode:

Function Name: $subStringMatching(char\ str1[n], char\ str2[m])$

1. Declare and initialize two variables, $temp = 0$ and $flog = 0$
2. For $i = 0$ and $str1[i] \neq '\backslash 0'$
3. $flag = 1$
4. For $j = 0$ and $str2[j] \neq '\backslash 0'$
5. If $(str1[i + j] \neq str2[j])$
6. $flag = 0$
7. Break
8. If $flag = 1$ then $temp++$
9. Return $temp$, i.e. the number of matches.

Pseudocode:

Function Name: $reverseString(char\ str[n])$

1. For $i = 0,\ j = strlen(str) - 1; i \leq j; i++, j--$
2. $str[i] \leftrightarrow str[j]$

Pseudocode:

Function Name: $stringCopy(char\ str1[n], char\ str2[m])$

1. For $i = 0$ and $str1[i] \neq '\backslash 0'$

2. $str2[i] = str1[i]$

Pseudocode:

Function Name: $stringCompare(char\ str1[n], char\ str2[m])$

1. For $i = 0$ and $str1[i] = str2[i]$ && $str1[i] = '\backslash 0'$
2. If ($str1[i] < str2[i]$) then print 'str1 is less than str2'.
3. If ($str1[i] > str2[i]$) then print 'str2 is less than str1'.
4. Else print 'str1 is equals to str2'.

Stick Data sheets of Program-8 here:

OUTPUT:

PROGRAM – 9

Structures

It is a user-defined data type used to group items of different types into a single type.

	Syntax

Declaration
(no memory is allocated)
	`struct structureName {`
	`  dataType memberName1;`
	`  dataType memberName2;`
	`  ....`
	`};`

Defining an instance
(creating variables of structure type)
	`struct structureName {`
	`  dataType memberName1;`
	`  dataType memberName2;`
	`  ....`
	`} variable1, varaible2, ...;`

	`// structure declared beforehand` `struct structureName variable1, variable2, .......;`

Accessing members	`// structure declared beforehand`
	`struct structureName variable1, variable2, .......;`
	`variable1.memberName1;`
	`variable1.memberName2;`
	`variable2.memberName1;`
	`.......;`

	`// structure declared beforehand`
	`struct structureName variable1, *variable2, .......;`
	`*variable2 = &variable1;`
	`variable2->memberName1;`
	`variable2->memberName2;`
	`variable1.memberName1;`
	`.......;`

Initializing members	`// structure declared beforehand`
	`struct structureName variable1;`
	`variable1.memberName1 = value1;`
	`variable1.memberName2 = value2;`
	`.......;`

	`// structure declared beforehand`
	`struct structureName variable1 = {value1, value2,`
	`.......;`

Typedef
(defining an alias for existing structure)
	`typedef struct structureName {`
	`  dataType memberName1;`
	`  dataType memberName2;`
	`  ....`
	`}strctNm;`

	`// structure declared beforehand` `typedef struct structureName strctNm;`

Nested Structures

Nested structures means inserting one structure into another as a member.

	Syntax

Declaration
(no memory is allocated)
	`struct parentStructure {`
	`  dataType member1;`
	`  struct childStructure{`
	`  dataType childMember1;`
	`  dataType childMember2;`
	`  ...`
	`  } chdStrct;`
	`  …`
	`};`

	`struct childStructure{`
	`  dataType childMember1;`
	`  dataType childMember2;`
	`  ...`
	`};`
	`struct parentStructure {`
	`  dataType member1;`
	`  struct childStructure chdStrct;`
	`  ...`
	`};`

Accessing members	`// structure declared beforehand`
	`struct parentStructure variable1, variable2, .......;`
	`variable1.member1;`
	`variable1.chdStrct.childMember1;`
	`variable2.member1;`
	`variable2.chdStrct.childMember2;`
	`.......;`

Self-Referential Structures

Structures containing references to the same type as themselves, i.e. one of the members is a pointer pointing to the same structure type.

Syntax:

```
struct structureName {
      dataType memberName1;
      dataType memberName2;
      struct structureName *memberName3;
      ....
};
```

Program for Lab 9

Write a program using structures to read, write and compute average-marks of n students. Take student name, USN, and marks in 5 subjects as input. Pass structure by reference to a function to update marks in one of the five subjects as given by user for a particular student. Finally display the new average for that student.

Stick Data sheets of Program-9 here:

OUTPUT:

PROGRAM – 10

File Handling in C

It helps to create, open, read, write, and close files. A file can be a text file (i.e. '.txt' files that contains data in the form of ASCII characters) or a binary file (i.e. '.bin' files that contains binary data in the form of 0's and 1's).

File Operations

- `FILE* pointerName;`

  Declares a file pointer which is a reference to a particular position in the opened file. The FILE macro is defined inside <stdio.h> header file.

- `FILE *fopen( const char *filename, const char *mode );`

  Opens a file indicated by `filename` and returns a pointer to the file stream associated with that file. `mode` is used to determine the file access mode. If successful, returns a pointer to the new file stream. On error, returns a null pointer.

File access mode string – Meaning (Explanation)	**Action if file already exists**	**Action if file does not exist**
"r" – read (Open a file for reading)	read from start	failure to open
"w" – write (Create a file for writing)	destroy contents	create new
"a" – append (Append to a file)	write to end	create new
"r+" – read extended (Open a file for read/write)	read from start	error
"w+" – write extended (Create a file for read/write)	destroy contents	create new
"a+" – append extended (Open a file for read/write)	write to end	create new
Note: * File access mode flag "b" can optionally be specified to open a file in binary mode. On the append file access modes, data is written to the end of the file regardless of the current position of the file position indicator. * File access mode flag "x" can optionally be appended to "w" or "w+" specifiers. This flag forces the function to fail if the file exists, instead of overwriting it. * When using fopen_s or freopen_s, file access permissions for any file created with "w" or "a" prevents other users from accessing it. File access mode flag "u" can optionally be prepended to any specifier that begins with "w" or "a", to enable the default fopen permissions.		

- `int fclose( FILE *stream );`
  Closes the given file stream. Returns 0 on success, EOF otherwise.

- `char* fgets( char* str, int count, FILE* stream );`
  Reads at most `count - 1` characters from the given file stream and stores them in the character array pointed to by `str`. Parsing stops if a newline character is found (in which case str will contain that newline character) or if end-of-file occurs. If bytes are read and no errors occur, writes a null character at the position immediately after the last character written to `str`. Returns `str` on success, null pointer on failure. Other functions to read from file are `fscanf()`, `fgetc()`, `fgetw()`, and `fread()`.

- `int fprintf( FILE* stream, const char* format, ... );`
  Writes the results to the output stream `stream`. Other functions to write to a file are `fputs()`, `fputc()`, `fputw()`, and `fwrite()`.

- `size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *file_pointer);`
  Writes data to a binary file. Here, ptr is pointer to the block of memory to be written; size is size of each element to be written (in bytes); nmemb is number of elements; and file_pointer is FILE pointer to the output file stream. Returns number of objects written.

- `size_t fread(void *ptr, size_t size, size_t nmemb, FILE *file_pointer);`
  Reads data from a binary file. Here, ptr is pointer to the block of memory to read; size is size of each element to read (in bytes); nmemb is number of elements; and file_pointer is FILE pointer to the input file stream. Returns number of objects read.

- `int fseek( FILE* stream, long offset, int origin );`
  Seeks the cursor to the given record in the file. Here, `stream` is file stream to modify; `offset` is number of characters to shift the position relative to `origin`; and `origin` is position to which `offset` is added. `origin` can have one of the following values: SEEK_SET, SEEK_CUR, SEEK_END.

- `rewind (file_pointer);`
  Used to bring the file pointer to the beginning of the file.

Programs for Lab 10

Write a program to demonstrate different file operations in C.

Stick Data sheets of Program-10 here:

OUTPUT:

APPENDIX

C Operator Precedence

Precedence	Operator	Description	Associativity
1	++ --	Suffix/postfix increment and decrement	Left-to-right
	()	Function call	
	[]	Array subscripting	
	.	Structure and union member access	
	->	Structure and union member access through pointer	
	(*type*){*list*}	Compound literal	
2	++ --	Prefix increment and decrement	Right-to-left
	+ -	Unary plus and minus	
	! ~	Logical NOT and bitwise NOT	
	(*type*)	Cast	
	*	Indirection (dereference)	
	&	Address-of	
	sizeof	Size-of	
	_Alignof	Alignment requirement	
3	* / %	Multiplication, division, and remainder	Left-to-right
4	+ -	Addition and subtraction	
5	<< >>	Bitwise left shift and right shift	
6	< <=	For relational operators < and ≤ respectively	
	> >=	For relational operators > and ≥ respectively	
7	== !=	For relational = and ≠ respectively	
8	&	Bitwise AND	
9	^	Bitwise XOR (exclusive or)	
10	|	Bitwise OR (inclusive or)	
11	&&	Logical AND	
12	||	Logical OR	
13	?:	Ternary conditional	Right-to-left
14	=	Simple assignment	
	+= -=	Assignment by sum and difference	
	*= /= %=	Assignment by product, quotient, and remainder	
	<<= >>=	Assignment by bitwise left shift and right shift	
	&= ^= |=	Assignment by bitwise AND, XOR, and OR	
15	,	Comma	Left-to-right

RV University Vision and Mission

Vision

To be a pioneering school of Computer Science and Engineering committed to fostering liberal education and empowering the next generation of technologists to make a positive global socio-economic impact.

Mission

- To be a pioneer in computer science education, fostering multidisciplinary research, innovation and entrepreneurship.
- To provide state-of-the-art facilities and dynamic curriculum that enables exemplary pedagogy, integrating advanced technology, theoretical foundations, and hands-on applications to address real-world challenges.
- To promote diverse communities of faculty and students through national and international academic, industry collaborations contributing to institution-building.
- To cultivate a generation of ethical, self-motivated, and empathetic problem solvers dedicated to achieving sustainable development goals.

RV UNIVERSITY
Go, change the world

RV Vidyanikethan Post, 8th Mile, Mysuru Road, Bengaluru - 560 059
For general enquiry: enquiry@rvu.edu.in | Phone: +91 95136 73778